

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
In [3]: %matplotlib inline
import matplotlib.pyplot as plt
from scipy import stats

# use seaborn plotting defaults
import seaborn as sns; sns.set()
```

Working with Perfectly Linear Dataset

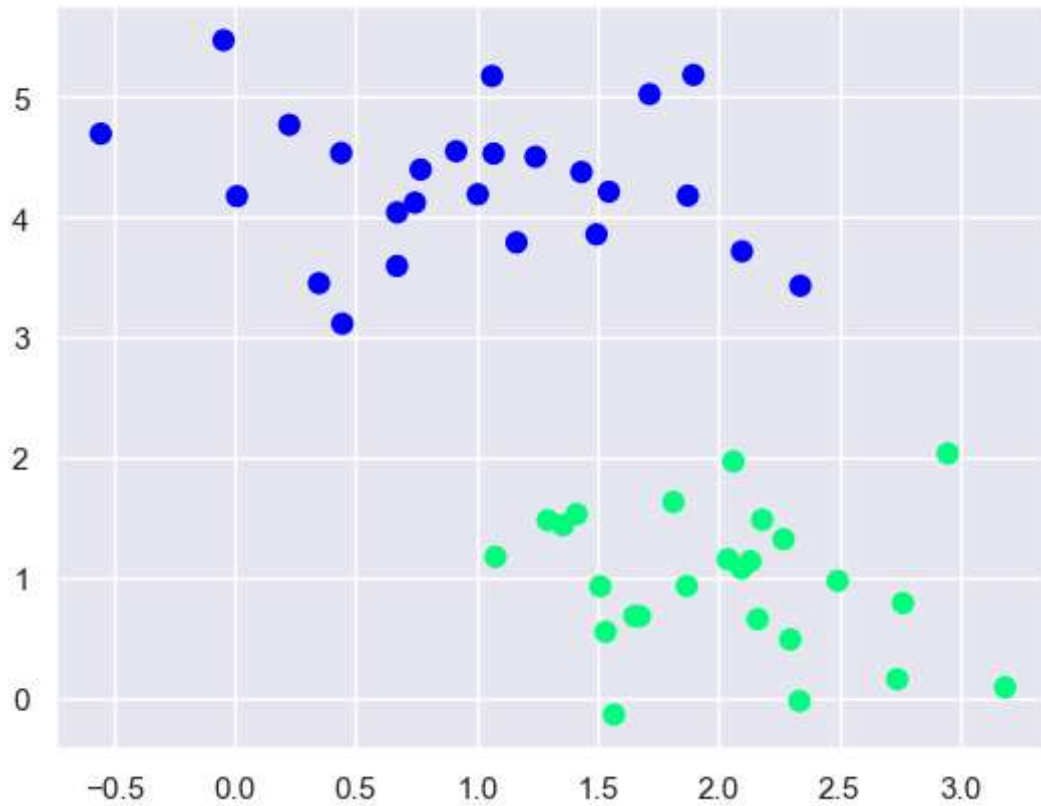
```
In [6]: from sklearn.datasets import make_blobs
X, y = make_blobs(n_samples=50, centers=2,
                  random_state=0, cluster_std=0.6)
```

```
In [8]: y
```

```
Out[8]: array([1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1,
               1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1,
               0, 1, 1, 0, 1, 0])
```

```
In [10]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='winter')
```

```
Out[10]: <matplotlib.collections.PathCollection at 0x11e951177a0>
```



```
In [12]: from sklearn.svm import SVC # "Support vector classifier"
model = SVC(kernel='linear', C=1)
model.fit(X, y)
```

```
Out[12]: SVC
SVC(C=1, kernel='linear')
```

```
In [14]: def plot_svc_decision_function(model, ax=None, plot_support=True):
    """Plot the decision function for a 2D SVC"""
    if ax is None:
        ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

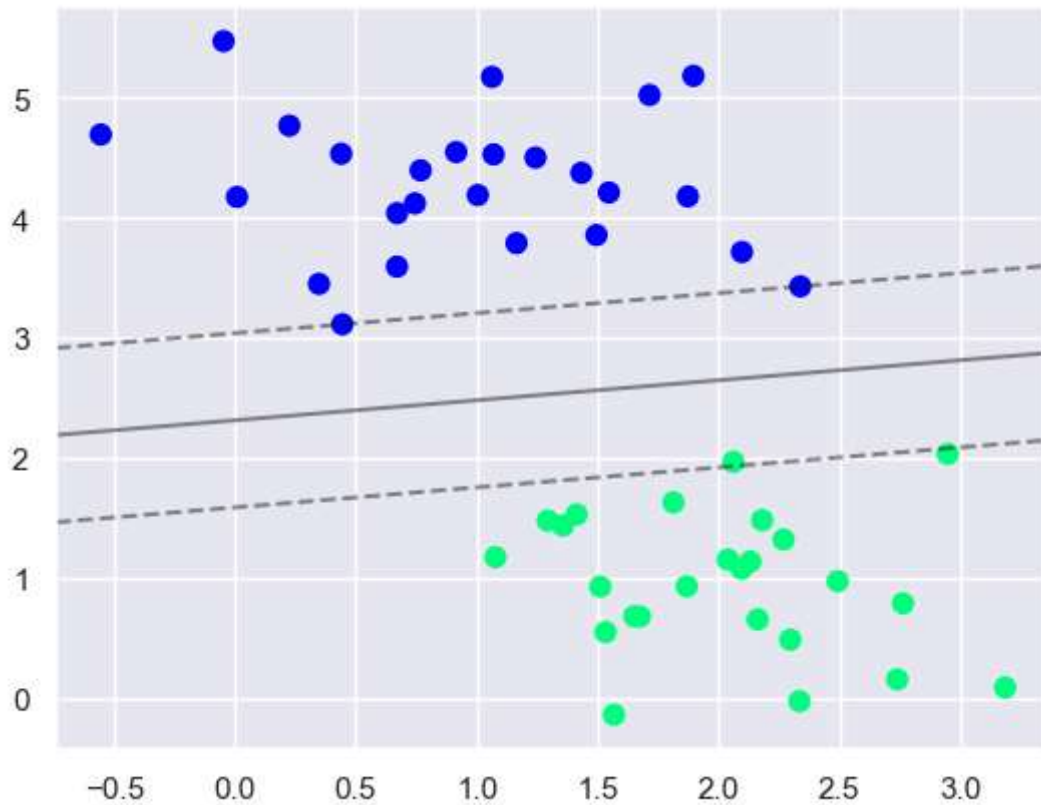
    # create grid to evaluate model
    x = np.linspace(xlim[0], xlim[1], 30)
    y = np.linspace(ylim[0], ylim[1], 30)
    Y, X = np.meshgrid(y, x)
    xy = np.vstack([X.ravel(), Y.ravel()]).T
    P = model.decision_function(xy).reshape(X.shape)

    # plot decision boundary and margins
    ax.contour(X, Y, P, colors='k',
               levels=[-1, 0, 1], alpha=0.5,
               linestyles=['--', '-', '--'])

    # plot support vectors
    if plot_support:
        model.support_vectors_
```

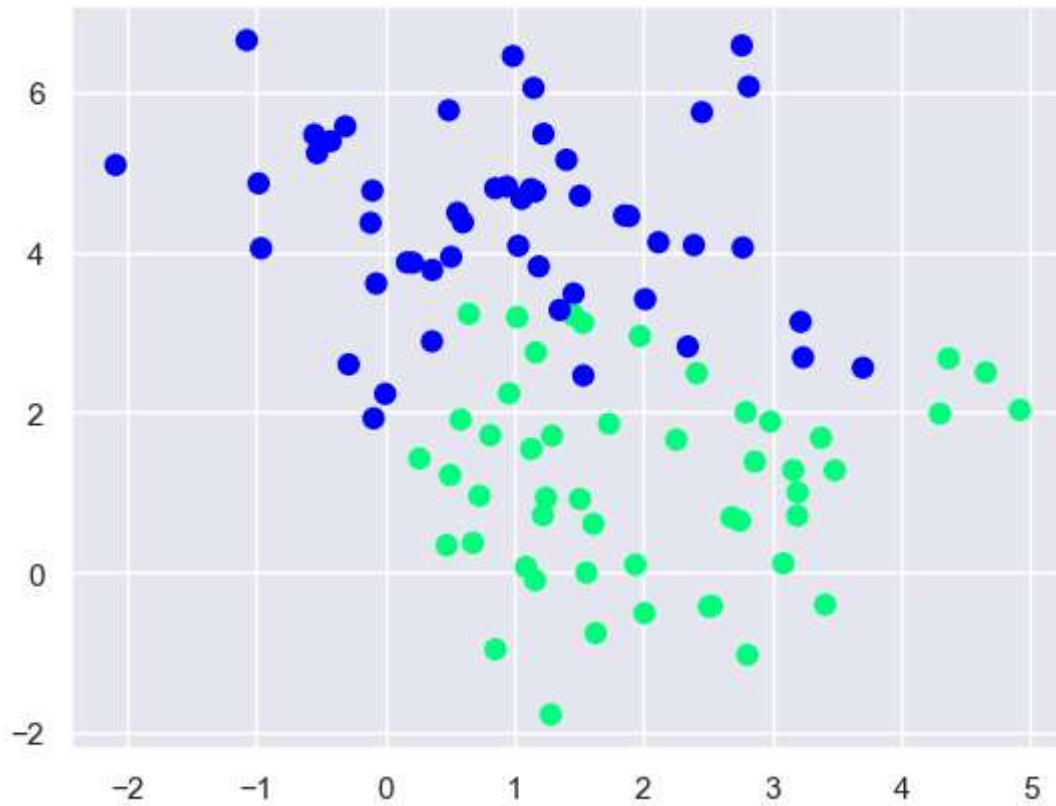
```
ax.scatter(model.support_vectors_[0],
           model.support_vectors_[1],
           s=300, linewidth=1, facecolors='none');
ax.set_xlim(xlim)
ax.set_ylim(ylim)
```

```
In [16]: plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='winter')
         plot_svc_decision_function(model);
```



Almost Linearly Separable Dataset

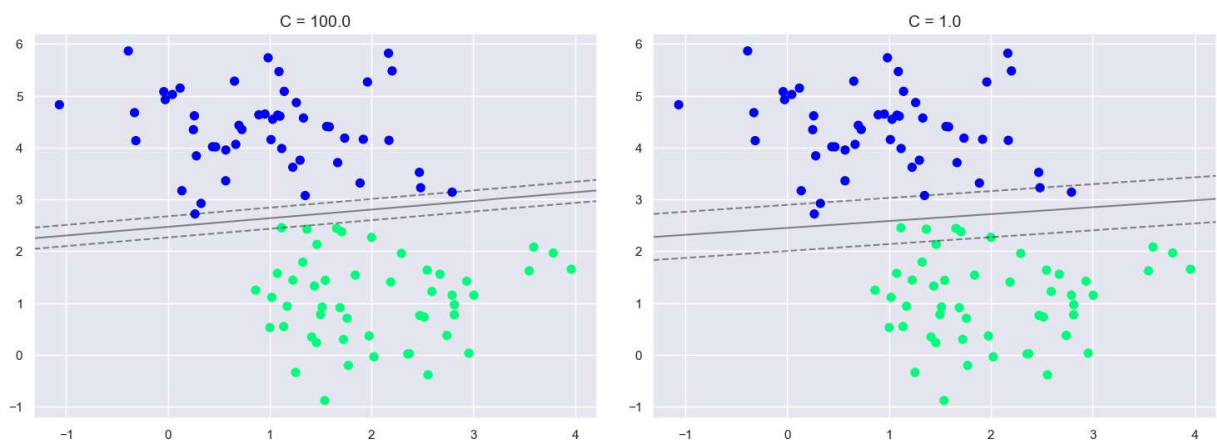
```
In [19]: X, y = make_blobs(n_samples=100, centers=2,
                           random_state=0, cluster_std=1.2)
         plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='winter');
```



```
In [21]: X, y = make_blobs(n_samples=100, centers=2,
                           random_state=0, cluster_std=0.8)

fig, ax = plt.subplots(1, 2, figsize=(16, 6))
fig.subplots_adjust(left=0.0625, right=0.95, wspace=0.1)

for axi, C in zip(ax, [100.0, 0.99]):
    model = SVC(kernel='linear', C=C).fit(X, y)
    axi.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='winter')
    plot_svc_decision_function(model, axi)
    axi.scatter(model.support_vectors_[:, 0],
                model.support_vectors_[:, 1],
                s=300, lw=1, facecolors='none');
    axi.set_title('C = {0:.1f}'.format(C), size=14)
```



```
In [23]: data = pd.read_csv('mushrooms.csv')
```

```
In [25]: data.shape
```

```
Out[25]: (8124, 23)
```

```
In [27]: data.head()
```

```
Out[27]:
```

	class	cap- shape	cap- surface	cap- color	bruises	odor	gill- attachment	gill- spacing	gill- size	gill- color	...	stalk- surface- below- ring
0	p	x	s	n	t	p	f	c	n	k	...	
1	e	x	s	y	t	a	f	c	b	k	...	
2	e	b	s	w	t	l	f	c	b	n	...	
3	p	x	y	w	t	p	f	c	n	n	...	
4	e	x	s	g	f	n	f	w	b	k	...	

5 rows × 23 columns



```
In [29]: X = data.drop(columns='class')
```

```
In [31]: y = data['class']
```

```
In [33]: from sklearn.preprocessing import LabelEncoder  
  
encoder = LabelEncoder()
```

```
In [35]: data.columns
```

```
Out[35]: Index(['class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor',  
              'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color',  
              'stalk-shape', 'stalk-root', 'stalk-surface-above-ring',  
              'stalk-surface-below-ring', 'stalk-color-above-ring',  
              'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number',  
              'ring-type', 'spore-print-color', 'population', 'habitat'],  
              dtype='object')
```

```
In [37]: df3 = pd.DataFrame()
```

```
In [39]: df3
```

```
Out[39]: —
```

```
In [41]: for col in data.columns:  
          df3[col] = encoder.fit_transform(data[col])
```

```
In [43]: df3
```

Out[43]:

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk
0	1	5	2	4	1	6	1	0	1	4	...	
1	0	5	2	9	1	0	1	0	0	4	...	
2	0	0	2	8	1	3	1	0	0	5	...	
3	1	5	3	8	1	6	1	0	1	5	...	
4	0	5	2	3	0	5	1	1	0	4	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
8119	0	3	2	4	0	5	0	0	0	11	...	
8120	0	5	2	4	0	5	0	0	0	11	...	
8121	0	2	2	4	0	5	0	0	0	5	...	
8122	1	3	3	4	0	8	1	0	1	0	...	
8123	0	5	2	4	0	5	0	0	0	11	...	

8124 rows × 23 columns



```
In [45]: X = df3.drop(columns='class')
```

```
In [47]: y = df3['class']
```

```
In [49]: df2 = pd.DataFrame()
df2['cap-shape'] = encoder.fit_transform(data['cap-shape'])
df2['cap-surface'] = encoder.fit_transform(data['cap-surface'])
df2['cap-color'] = encoder.fit_transform(data['cap-color'])
df2['bruises'] = encoder.fit_transform(data['bruises'])
df2['odor'] = encoder.fit_transform(data['odor'])
df2['gill-attachment'] = encoder.fit_transform(data['gill-attachment'])
df2['gill-spacing'] = encoder.fit_transform(data['gill-spacing'])
df2['gill-size'] = encoder.fit_transform(data['gill-size'])
df2['gill-color'] = encoder.fit_transform(data['gill-color'])
df2['stalk-shape'] = encoder.fit_transform(data['stalk-shape'])
df2['stalk-root'] = encoder.fit_transform(data['stalk-root'])
df2['stalk-surface-above-ring'] = encoder.fit_transform(data['stalk-surface-above-ring'])
df2['stalk-surface-below-ring'] = encoder.fit_transform(data['stalk-surface-below-ring'])
df2['stalk-color-above-ring'] = encoder.fit_transform(data['stalk-color-above-ring'])
df2['stalk-color-below-ring'] = encoder.fit_transform(data['stalk-color-below-ring'])
df2['veil-type'] = encoder.fit_transform(data['veil-type'])
df2['veil-color'] = encoder.fit_transform(data['veil-color'])
df2['ring-number'] = encoder.fit_transform(data['ring-number'])
df2['ring-type'] = encoder.fit_transform(data['ring-type'])
df2['spore-print-color'] = encoder.fit_transform(data['spore-print-color'])
df2['population'] = encoder.fit_transform(data['population'])
```

```
df2['habitat'] = encoder.fit_transform(data['habitat'])
df2['class'] = data['class']
```

```
In [51]: X = df2.drop(columns='class')
y = df2['class']
```

```
In [53]: x_train , x_test, y_train,y_test = train_test_split(X,y,random_state = 20)
```

```
In [55]: from sklearn.svm import SVC
```

```
In [57]: svc= SVC()
svc.fit(x_train,y_train)
```

```
Out[57]: SVC
```

```
SVC()
```

```
In [59]: svc.score(x_test,y_test)
```

```
Out[59]: 0.9911373707533235
```

```
In [ ]:
```

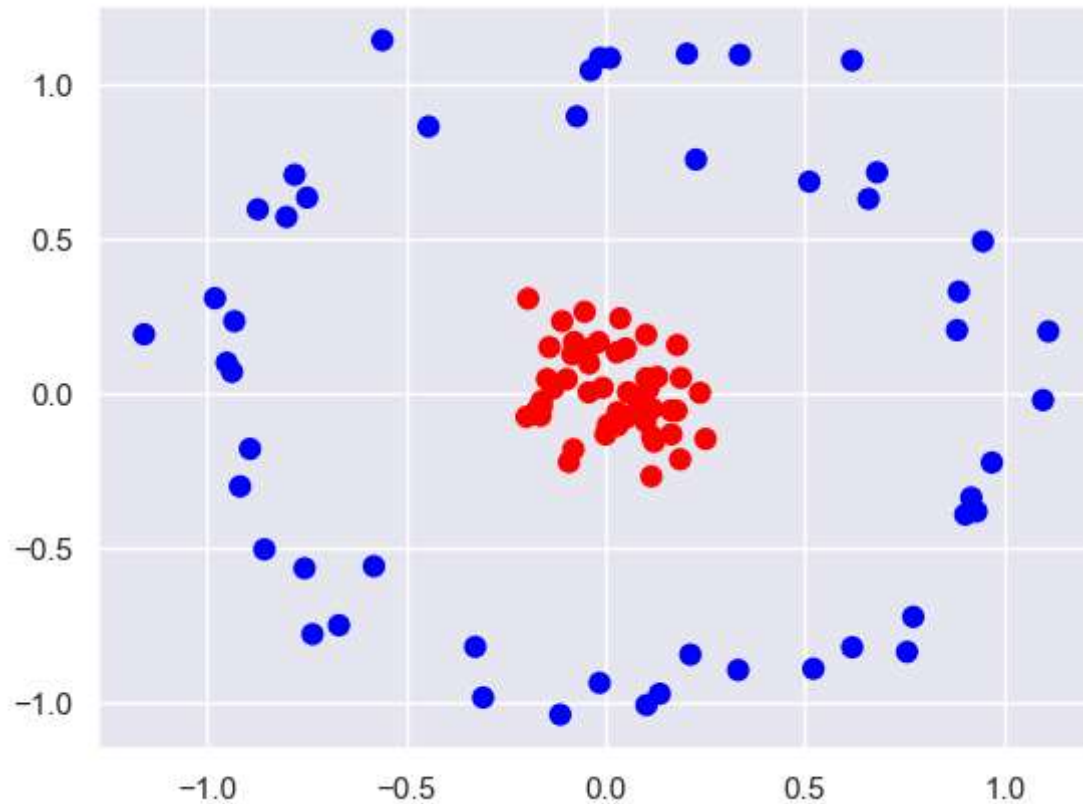
```
In [62]: #sns.pairplot(df2, hue='class')
```

```
In [64]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.axes._axes import _log as matplotlib_axes_logger
from mpl_toolkits import mplot3d
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from matplotlib.colors import ListedColormap
```

```
In [66]: from sklearn.datasets import make_circles
X, y = make_circles(100, factor=.1, noise=.1)

plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='bwr')
```

```
Out[66]: <matplotlib.collections.PathCollection at 0x11e9ad887a0>
```



```
In [68]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20)
```

```
In [70]: classifier = SVC(kernel="linear")
classifier.fit(X_train, y_train.ravel())
y_pred = classifier.predict(X_test)
```

```
In [72]: from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

```
Out[72]: 0.7
```

```
In [74]: zero_one_colormap = ListedColormap(['blue', 'red'])
def plot_decision_boundary(X, y, clf):
    X_set, y_set = X, y
    X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1,
                                   stop = X_set[:, 0].max() + 1,
                                   step = 0.01),
                          np.arange(start = X_set[:, 1].min() - 1,
                                   stop = X_set[:, 1].max() + 1,
                                   step = 0.01))

    plt.contourf(X1, X2, clf.predict(np.array([X1.ravel(),
                                              X2.ravel()]).T).reshape(X1.shape),
                 alpha = 0.75,
                 cmap = zero_one_colormap)
    plt.xlim(X1.min(), X1.max())
    plt.ylim(X2.min(), X2.max())
    for i, j in enumerate(np.unique(y_set)):
        plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
```



```

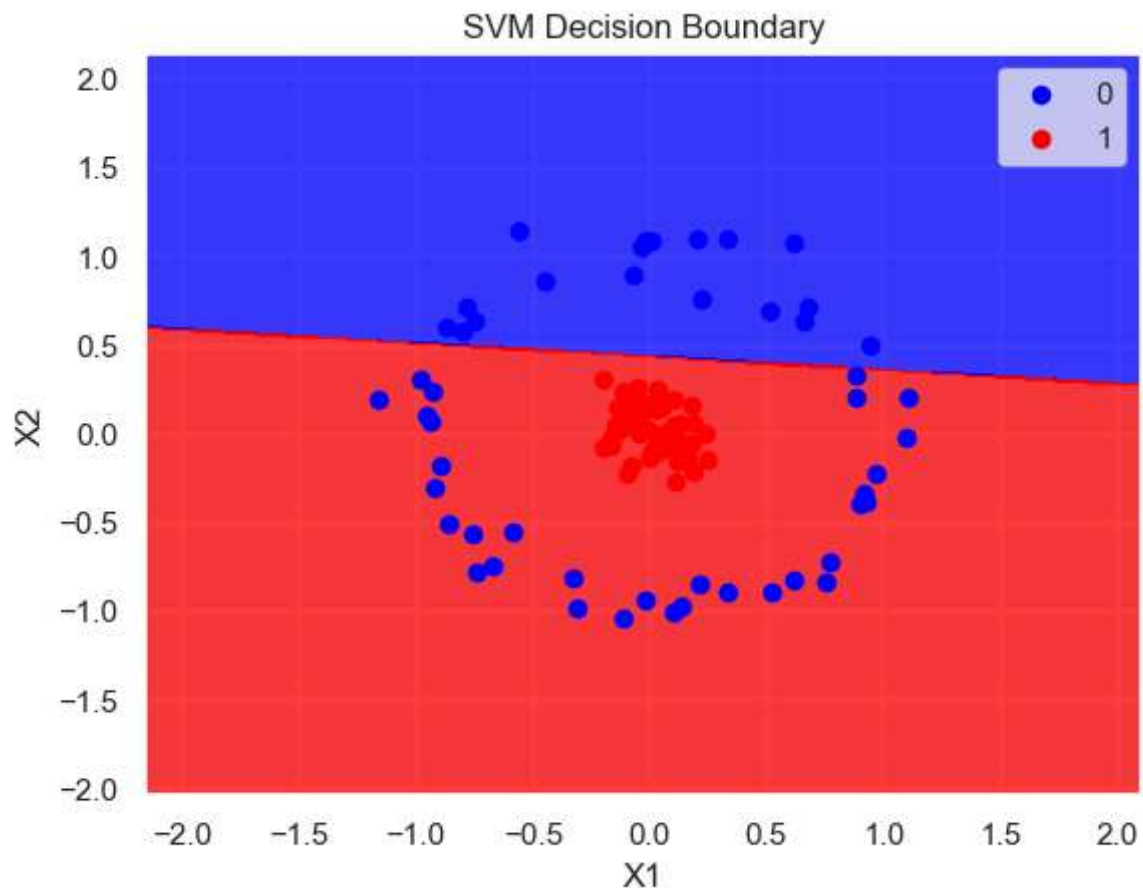
        c = (zero_one_colourmap)(i), label = j)
    plt.title('SVM Decision Boundary')
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    return plt.show()

```

In [76]: `plot_decision_boundary(X, y, classifier)`

C:\Users\SHREE\AppData\Local\Temp\ipykernel_9984\3603277588.py:18: UserWarning: *c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2D array with a single row if you intend to specify the same RGB or RGBA value for all points.

```
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
```



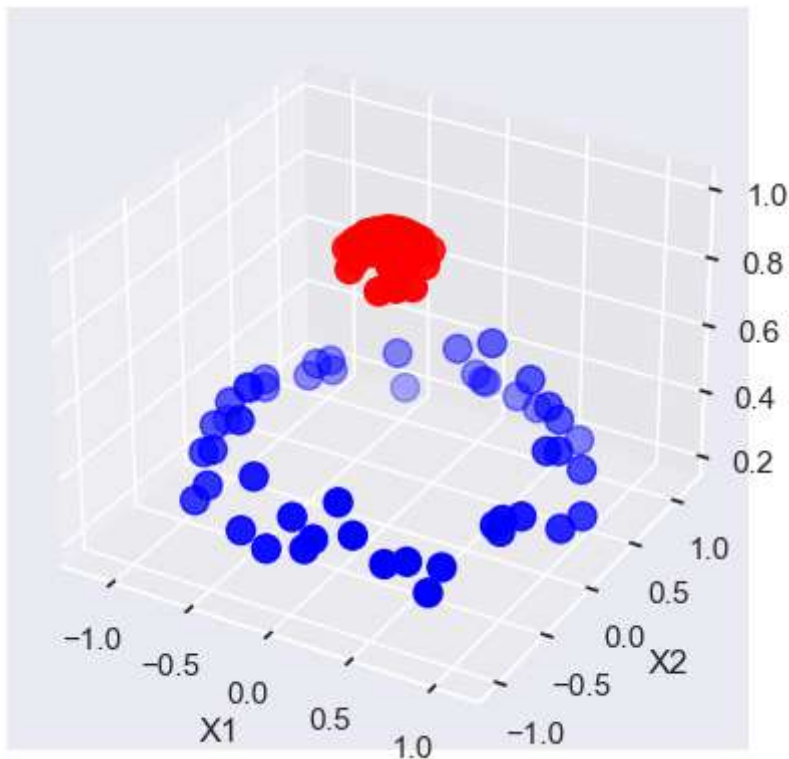
```

In [82]: def plot_3d_plot(X, y):
    r = np.exp(-(X ** 2).sum(1))
    ax = plt.subplot(projection='3d')
    ax.scatter3D(X[:, 0], X[:, 1], r, c=y, s=100, cmap='bwr')
    ax.set_xlabel('X1')
    ax.set_ylabel('X2')
    ax.set_zlabel('y')
    return ax
    rbf_classifier = SVC(kernel="rbf")
    rbf_classifier.fit(X_train, y_train)
    y_pred = rbf_classifier.predict(X_test)

```

```
In [84]: plot_3d_plot(X,y)
```

```
Out[84]: <Axes3D: xlabel='X1', ylabel='X2', zlabel='y'>
```



```
In [86]: rbf_classifier = SVC(kernel="rbf")
rbf_classifier.fit(X_train, y_train)
y_pred = rbf_classifier.predict(X_test)
```

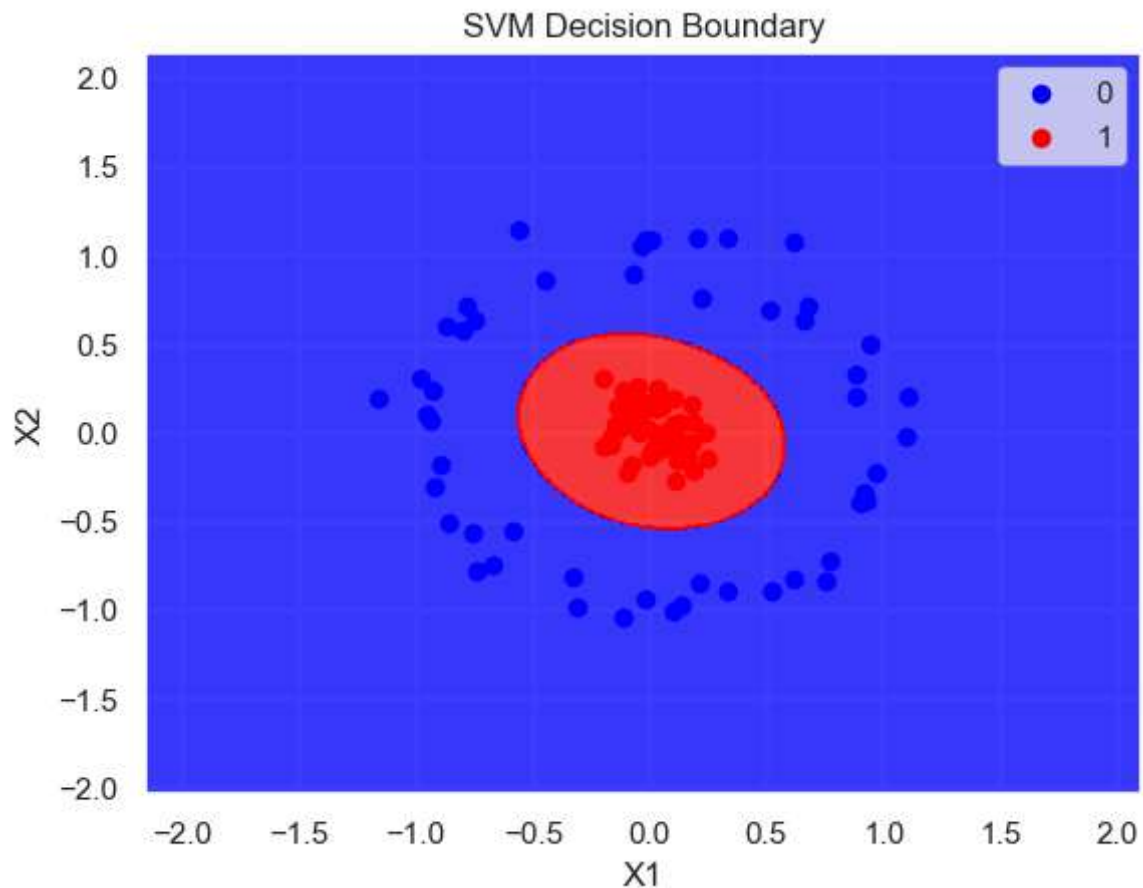
```
In [88]: accuracy_score(y_test, y_pred)
```

```
Out[88]: 1.0
```

```
In [90]: plot_decision_boundary(X, y, rbf_classifier)
```

C:\Users\SHREE\AppData\Local\Temp\ipykernel_9984\3603277588.py:18: UserWarning: *c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2D array with a single row if you intend to specify the same RGB or RGBA value for all points.

```
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
```



```
In [92]: poly_classifier = SVC(kernel="poly",degree=2)
poly_classifier.fit(X_train, y_train)
y_pred = poly_classifier.predict(X_test)
```

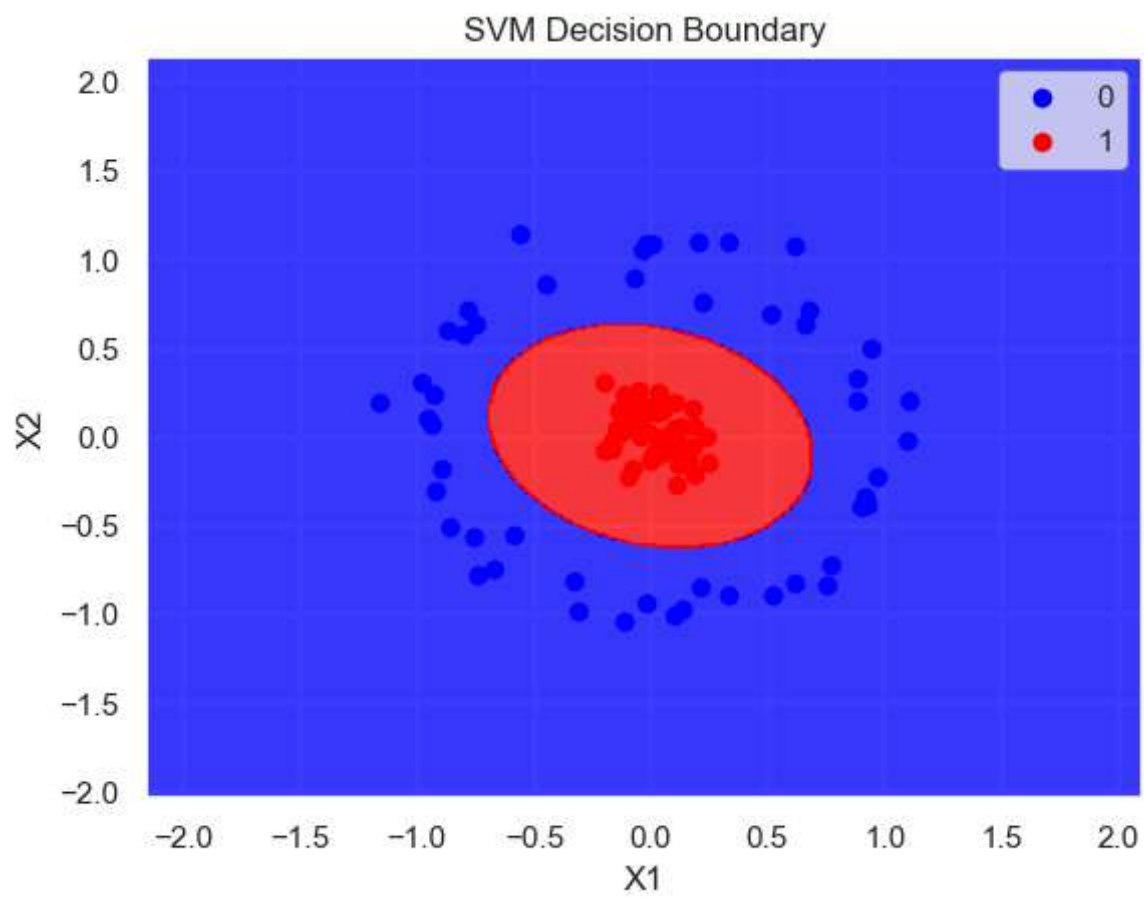
```
In [94]: accuracy_score(y_test, y_pred)
```

```
Out[94]: 1.0
```

```
In [96]: plot_decision_boundary(X, y, poly_classifier)
```

C:\Users\SHREE\AppData\Local\Temp\ipykernel_9984\3603277588.py:18: UserWarning: *c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case its length matches with *x* & *y*. Please use the *color* keyword-argument or provide a 2D array with a single row if you intend to specify the same RGB or RGBA value for all points.

```
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
```



In [98]: x

```
Out[98]: array([[ -1.98155398e-01,  -7.76252072e-02],
 [  1.11044030e+00,   1.99033881e-01],
 [  2.26665521e-01,   7.55305053e-01],
 [  2.36729953e-01,   4.84363439e-04],
 [  2.12762324e-01,  -8.46910456e-01],
 [  5.63482200e-02,   1.97162309e-03],
 [  2.81739900e-02,   1.31046648e-01],
 [-8.55608032e-01,  -5.06432265e-01],
 [-7.55060930e-01,  -5.67597015e-01],
 [-8.72075778e-01,   5.94080716e-01],
 [  1.00454220e-01,  -9.29759520e-02],
 [  1.12187148e-02,   1.08359328e+00],
 [-7.07809949e-03,   1.66528109e-02],
 [  9.30278151e-01,  -3.84224143e-01],
 [-1.94221911e-01,   3.04757425e-01],
 [-1.49898931e-02,  -9.38517983e-01],
 [  9.68895180e-01,  -2.25318257e-01],
 [  1.77715299e-01,  -5.77066403e-02],
 [  9.01880227e-01,  -3.93881584e-01],
 [-1.40772847e-01,   1.48021233e-01],
 [-4.12159332e-02,   9.50221744e-02],
 [  1.87125693e-01,  -2.14916382e-01],
 [  2.99435298e-02,  -6.26037707e-02],
 [  1.08491223e-01,   1.55096381e-02],
 [  6.18242130e-01,   1.07583506e+00],
 [-7.88087452e-02,   1.64472434e-01],
 [  9.27547599e-02,  -2.86003610e-02],
 [  5.06089419e-02,   1.43664421e-01],
 [-5.59768586e-01,   1.14143304e+00],
 [  5.11034828e-01,   6.83491268e-01],
 [  1.29546698e-01,   5.13857588e-02],
 [-6.68601540e-01,  -7.52021706e-01],
 [-7.34954523e-01,  -7.80925476e-01],
 [-4.21395301e-02,   1.64768501e-03],
 [-1.60108075e-01,  -2.70505261e-02],
 [-9.18747177e-02,  -2.23878587e-01],
 [  1.16740312e-01,  -1.41396623e-01],
 [  3.36818319e-01,   1.09437617e+00],
 [-8.91689899e-01,  -1.81266847e-01],
 [-9.50238986e-01,   9.76843164e-02],
 [-8.00185916e-01,   5.69239883e-01],
 [  6.18544190e-01,  -8.23386946e-01],
 [-8.33390265e-02,   1.24447123e-01],
 [-4.44111634e-01,   8.61851666e-01],
 [  1.09708429e+00,  -2.33053606e-02],
 [  1.16402882e-01,  -5.11639410e-02],
 [-5.26503513e-02,   2.62561111e-01],
 [  1.21464207e-01,  -1.57996829e-01],
 [  1.79723474e-01,   1.54864127e-01],
 [  9.17151676e-01,  -3.38951175e-01],
 [  7.56034016e-01,  -8.38223799e-01],
 [-1.60164514e-01,  -4.45495310e-02],
 [-9.79619720e-01,   3.06470691e-01],
 [-1.46561466e-01,   4.47159633e-02],
 [  2.04175920e-01,   1.09781245e+00],
 [  5.55465223e-03,  -1.03727940e-01],
```

```

[-3.73673563e-02,  1.04425922e+00],
[ 2.50707793e-01, -1.49053243e-01],
[-2.50302497e-04, -1.34676000e-01],
[ 4.15316490e-02, -7.35949555e-02],
[ 6.59597855e-01,  6.27834021e-01],
[-1.63208612e-01, -7.40156440e-02],
[ 1.01814130e-01,  1.87969594e-01],
[ 1.65032737e-01, -1.33955900e-01],
[ 1.02221385e-01,  4.75588452e-02],
[ 6.80866267e-01,  7.13755688e-01],
[-1.77378186e-02,  1.64480013e-01],
[ 9.46334947e-01,  4.90636152e-01],
[-9.30877350e-01,  2.32022108e-01],
[-9.16705108e-01, -3.02565678e-01],
[-1.31125895e-01,  1.51645116e-02],
[-3.26787760e-01, -8.21830139e-01],
[ 1.02636784e-01, -1.01024769e+00],
[-7.79961554e-01,  7.05825970e-01],
[ 5.21183468e-01, -8.92709726e-01],
[-3.07073312e-01, -9.85958561e-01],
[ 3.65946295e-02,  2.41200805e-01],
[-1.38207378e-02,  1.08403866e+00],
[ 8.85851871e-01,  3.27709431e-01],
[ 8.81303090e-01,  2.03063081e-01],
[ 7.71636521e-01, -7.24633643e-01],
[-8.06567556e-02, -1.82571339e-01],
[-9.73053974e-02,  4.44813195e-02],
[-5.80888885e-01, -5.60889519e-01],
[ 3.32478892e-01, -8.96823907e-01],
[-1.15731433e+00,  1.89623915e-01],
[-9.36998649e-01,  6.82793870e-02],
[ 1.35840647e-01, -9.73600164e-01],
[-1.71120432e-01, -5.44202945e-02],
[-1.09042399e-01,  2.31902743e-01],
[ 1.89014793e-01,  4.85638331e-02],
[-1.13666629e-01, -1.04124495e+00],
[ 8.27552261e-02, -3.08977943e-02],
[ 1.63981388e-01, -5.68169370e-02],
[ 1.14770803e-01, -2.70048410e-01],
[-4.31342641e-02,  1.41764090e-01],
[-7.19269168e-02,  8.94989599e-01],
[ 5.15662838e-02, -8.02828755e-02],
[-7.48440142e-01,  6.31834619e-01],
[ 3.02363863e-02, -1.05383197e-01]]))

```

In [100...

```
np.exp(-(X**2)).sum(1)
```

```
Out[100...] array([1.95548778, 1.25255429, 1.51517105, 1.94550007, 1.44383059,
      1.99682603, 1.98217994, 1.25469022, 1.2900369 , 1.17005014,
      1.98135242, 1.30894819, 1.99967262, 1.28362766, 1.87428597,
      1.41421991, 1.34161221, 1.96558628, 1.29964644, 1.95870607,
      1.98931412, 1.92045137, 1.99519222, 1.98805813, 0.99663833,
      1.96711985, 1.99061581, 1.97701408, 1.00275161, 1.39693991,
      1.98072068, 1.20758121, 1.12609141, 1.99822312, 1.97395972,
      1.9427083 , 1.96666967, 1.19465196, 1.41920745, 1.39587353,
      1.2503612 , 1.18973625, 1.97771089, 1.29678299, 1.29957069,
      1.98392743, 1.93061601, 1.96070075, 1.94451797, 1.322673 ,
      1.0599148 , 1.97269087, 1.29337677, 1.97675127, 1.25880359,
      1.98926734, 1.33466071, 1.9171083 , 1.98202581, 1.99287503,
      1.32145547, 1.96825125, 1.95497177, 1.9553476 , 1.9873459 ,
      1.22985603, 1.97299442, 1.19444167, 1.36799587, 1.34407955,
      1.98272304, 1.40766453, 1.34989949, 1.15188286, 1.21284475,
      1.28829756, 1.94214388, 1.30858481, 1.35441346, 1.41952745,
      1.14282916, 1.96073271, 1.98859971, 1.44368357, 1.34275261,
      1.22669055, 1.41097602, 1.36927161, 1.96818518, 1.93582179,
      1.9625484 , 1.32533825, 1.99222075, 1.97024546, 1.91658353,
      1.97824471, 1.44371687, 1.99091984, 1.24196263, 1.988042  ])
```

```
In [102...] X_new=np.exp(-(X**2))
```

```
In [104...] plt.scatter(X_new[:, 0], X_new[:, 1], c=y, s=50, cmap='bwr')
```

```
Out[104...] <matplotlib.collections.PathCollection at 0x11e9dba97f0>
```

